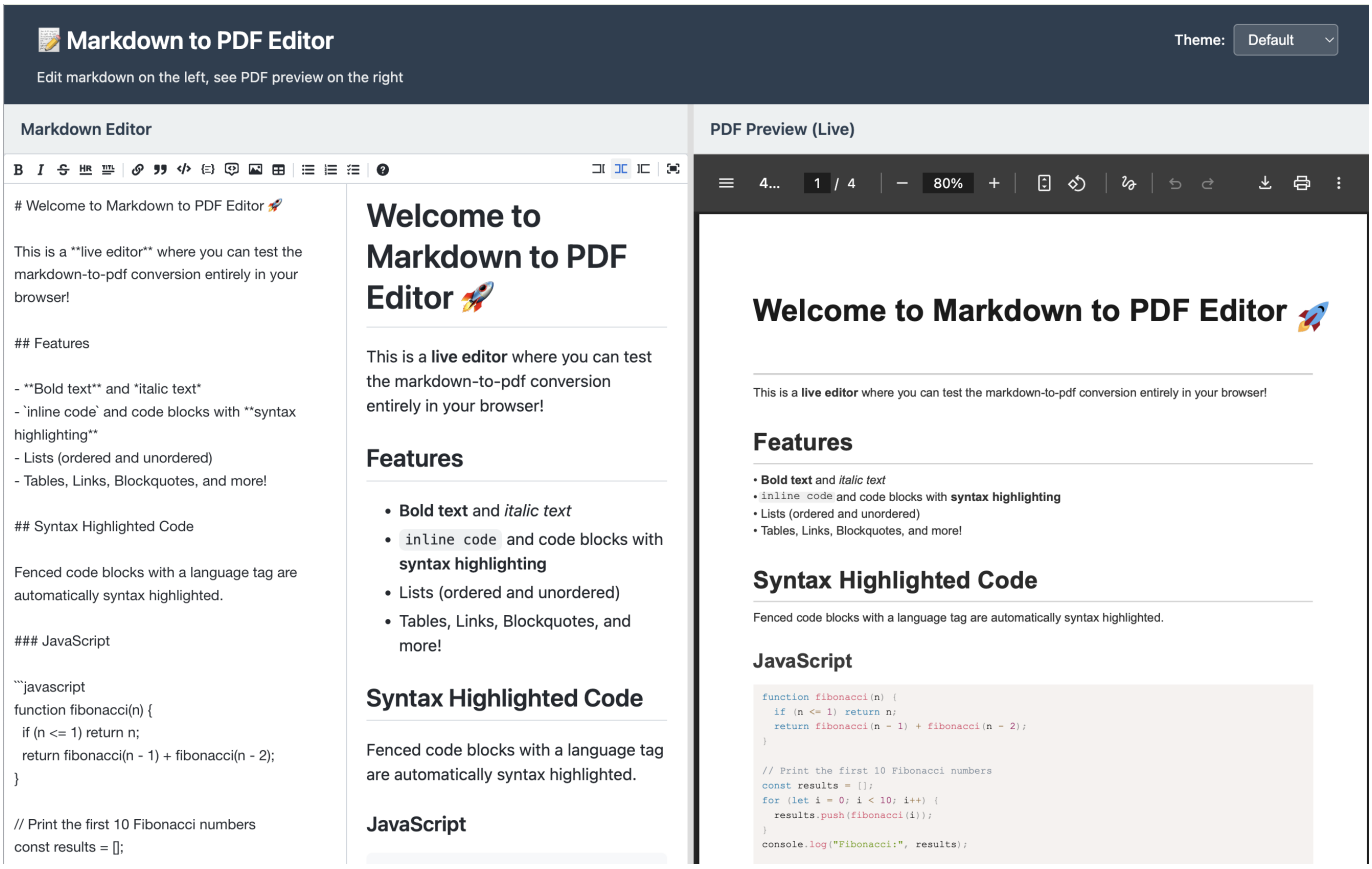
Basic-Markdown-To-PDF

A lightweight TypeScript library that converts Markdown files into styled PDF documents. Built on [marked](#) for parsing and [PDFKit](#) for PDF generation. [README.pdf](#) | [Live Demo](#)



Features

- **Headings** (h1–h6) with configurable fonts, sizes, and colors
- **Inline formatting** — bold, italic, bold-italic, strikethrough, inline code
- **Code blocks** with monospace font and background shading
- **Blockquotes** with colored left border
- **Lists** — ordered and unordered, with nested sub-lists
- **Tables** with header row highlighting and cell borders
- **Links** rendered as clickable PDF hyperlinks
- **Images** — local (PNG, JPEG) and remote (HTTP/HTTPS) with automatic SVG-to-PNG conversion via [@resvg/resvg-js](#)
- **Mermaid diagrams** — render `mermaid` fenced code blocks as diagrams (flowchart, sequence, class, state, ER, gantt, pie, mindmap) via [@speajus/mermaid-to-svg](#)
- **Horizontal rules**

- **Automatic page breaks** when content exceeds the current page
- **Fully themeable** — customize fonts, colors, spacing, page size, and margins

Installation

```
npm install @speajus/markdown-to-pdf
```

Quick Start

CLI

Convert a Markdown file to PDF from the command line:

```
# Run directly with npx (no install needed)
npx @speajus/markdown-to-pdf <input.md> [output.pdf]

# Or if installed globally / as a project dependency
markdown-to-pdf <input.md> [output.pdf]
```

If the output path is omitted, the PDF is written alongside the input file with a `.pdf` extension.

```
# Converts README.md    README.pdf
npx @speajus/markdown-to-pdf README.md

# Explicit output path
npx @speajus/markdown-to-pdf docs/report.md output/report.pdf
```

Programmatic API

```
import { generatePdf } from '@speajus/markdown-to-pdf';

// Returns a PDF Buffer
const markdown = '# Hello World\n\nThis is a **test**.';
const pdfBuffer = await generatePdf(markdown);

// With options
import { generatePdf, createNodeImageRenderer } from '@speajus/markdown-to-pdf';

const buffer = await generatePdf(markdown, {
  basePath: '/path/to/markdown/directory',
  renderImage: createNodeImageRenderer('/path/to/markdown/directory'),
});
```

Generate Sample PDFs

The `samples/` directory contains example Markdown files. Generate PDFs for all of them at once:

```
npm run generate
```

Output is written to the `output/` directory.

Configuration

Both `generatePdf` and `renderMarkdownToPdf` accept an optional `PdfOptions` object:

```
interface PdfOptions {
  theme?: ThemeConfig;     // Typography, colors, and component styles
  pageLayout?: PageLayout; // Page size and margins
  basePath?: string;       // Base directory for resolving relative image paths
  syntaxHighlight?: boolean; // Enable syntax highlighting (default: true)
  lineNumbers?: boolean;   // Show line numbers in code blocks (default: false)
  languages?: string[];    // Prism.js languages to load (default: all)
  zebraStripes?: boolean;  // Zebra-stripe table rows (default: true)
}
```

Mermaid Diagrams

Fenced code blocks with the language `mermaid` are automatically rendered as diagrams:

```mermaid
flowchart TD
    A[Start] --> B{Decision}
    B -->|Yes| C[Process A]
    B -->|No| D[Process B]
    C --> E[End]
    D --> E
```

Mermaid theme colors are integrated with the PDF theme. Each built-in theme includes matching mermaid colors. You can also customize them:

```
await generatePdf(markdown, {
  theme: {
    ...defaultTheme,
    // Use a built-in mermaid theme
    mermaid: 'forest', // 'default' | 'dark' | 'forest' | 'neutral'
  },
});

// Or provide custom mermaid colors
await generatePdf(markdown, {
  theme: {
    ...defaultTheme,
    mermaid: {
      background: '#ffffff',
      primaryColor: '#4a90d9',
      secondaryColor: '#b8d4f0',
      lineColor: '#333333',
      primaryTextColor: '#1a1a1a',
      borderColor: '#4a90d9',
    },
  },
});
```

Supported diagram types: flowchart, sequence, class, state, ER, gantt, pie, and mindmap.

Page Layout

```
import { generatePdf } from '@speajus/markdown-to-pdf';

await generatePdf(markdown, {
  pageLayout: {
    pageSize: 'A4',
    margins: { top: 72, right: 72, bottom: 72, left: 72 },
  },
});
```

The default layout uses **Letter** page size with 50pt margins on all sides.

Theming

Override any part of the default theme to customize the look of the generated PDF:

```
import { generatePdf, defaultTheme } from '@speajus/markdown-to-pdf';

await generatePdf(markdown, {
  theme: {
    ...defaultTheme,
    headings: {
      ...defaultTheme.headings,
      h1: { font: 'Helvetica-Bold', fontSize: 32, color: '#0a3d62', bold: true },
    },
    linkColor: '#e74c3c',
  },
});
```

The full `ThemeConfig` interface exposes styles for:

Section	Configurable properties
headings	Font, size, and color for each level (h1–h6)
body	Font, size, color, and line gap
code.inline	Font, size, color, and background color
code.block	Font, size, color, background color, and padding
blockquote	Border color, border width, italic flag, and indent
table	Header background, border color, cell padding, and zebra color
linkColor	Color for hyperlink text
horizontalRuleColor	Color for --- dividers
mermaid	Diagram colors: primary, secondary, text, line, border, background

Project Structure

```
% % %  src/
%    % % %  index.ts              # Public API — generatePdf, renderMarkdownToPdf, exports
%    % % %  cli.ts                # Command-line entry point
%    % % %  renderer.ts           # Markdown-to-PDF rendering engine
%    % % %  mermaid-renderer.ts   # Mermaid diagram PNG rendering
%    % % %  styles.ts             # Default theme and page layout
%    % % %  themes/index.ts       # Built-in theme variants
%    % % %  types.ts              # TypeScript interfaces for options and theming
% % %  samples/
%    % % %  generate.ts   # Script to batch-generate sample PDFs
%    % % %  sample.md     # Full-featured sample document
%    % % %  mermaid.md    # Mermaid diagram examples (all diagram types)
%    % % %  image.md      # Image rendering tests (local, remote, SVG, broken)
%    % % %  logo.svg      # Sample SVG image
%    % % %  logo.png      # Sample PNG image
%    % % %  sample.png    # Sample raster image
% % %  output/           # Generated PDF output directory
% % %  package.json
% % %  tsconfig.json
```

Dependencies

Package	Purpose
marked	Markdown parsing and tokenization
pdfkit	PDF document generation
@resvg/resvg-js	SVG-to-PNG rasterization for image embedding
@speajus/mermaid-to-svg	Mermaid diagram rendering (parse layout SVG)

Scripts

Command	Description
npm run build	Compile TypeScript to dist/
npm run generate	Generate sample PDFs from samples/*.md into output/

License

ISC